

PROYECTO DE APRENDIZAJE AUTOMÁTICO

# Clasificación de Solicitudes de Préstamo

Análisis comparativo de complejidad y desbalanceo

Autor: Pablo Aguilar García

Curso 2025-2026

---

# CONTEXTO DEL PROBLEMA

## Objetivo de Negocio

Automatizar la decisión de aprobación de préstamos (loan\_status) basándose en el riesgo crediticio del solicitante.

- ✓ **Clasificación Binaria:** Aprobado (1) vs Rechazado (0).
- ✓ **Escalabilidad:** Comparación de modelos desde Regresión Logística hasta Redes Profundas.
- ✓ **Realismo:** Dataset de Kaggle con 45.000 ejemplos reales.



# VARIABLES PREDICTORAS



## Perfil Personal

Edad, Género, Nivel  
Educativo, Tipo de Vivienda.



## Finanzas

Ingresos anuales, Experiencia  
laboral, Cantidad solicitada.



## Historial

Credit Score, Defaults  
previos, Ratio  
ingresos/préstamo.

# ESTRATEGIA DE PREPROCESADO

## Prevención de Data Leakage

- 🛡️ **División previa:** Split Train/Test antes de cualquier transformación.
- 🛡️ **Normalización Z-Score:** Estadísticos calculados solo en Train.
- 🛡️ **Codificación:** One-Hot Encoding para variables categóricas.
- 🛡️ **Validación Cruzada:** Stratified K-Fold ( $k=5$ ) para estabilidad.



# LIMPIEZA CRÍTICA DE OUTLIERS

| Variable                     | Regla Aplicada   | Razón Técnica                              |
|------------------------------|------------------|--------------------------------------------|
| Edad (person_age)            | $\leq 100$ años  | Eliminación de registros no plausibles.    |
| Experiencia (person_emp_exp) | $\leq 60$ años   | Consistencia con la vida laboral máxima.   |
| Ingresos (person_income)     | $\leq 1.000.000$ | Evitar distorsiones de escala extremas.    |
| Resultado                    | -31 registros    | Limpieza conservadora para preservar info. |

# TÉCNICAS APLICADAS



## Regresión Logística

**Base Line:** Implementada en NumPy. Gradiente descendente, regularización L2.



## Red Neuronal (NN)

**NumPy:** 1 capa oculta (8 neuronas), Sigmoide, Backpropagation manual.



## Red Profunda (DNN)

**PyTorch:** Arquitectura 32-16-8, activaciones ReLU, Optimizador Adam.

# METODOLOGÍA EXPERIMENTAL

**K=5**

Folds Estratificados

**F1**

Métrica Principal

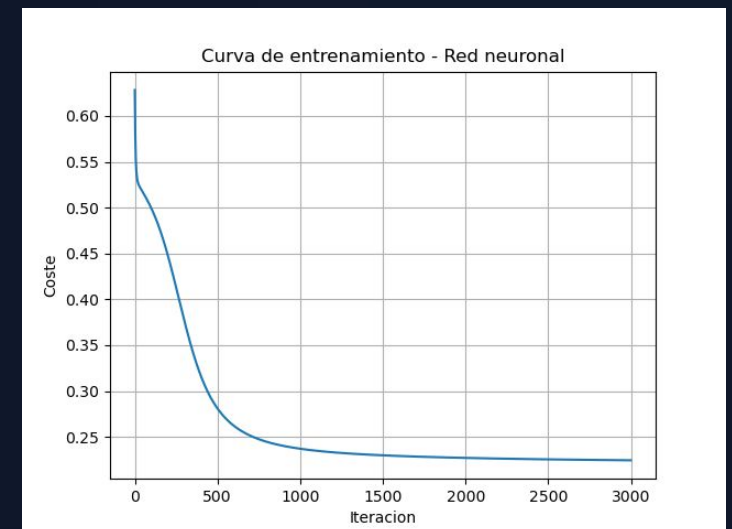
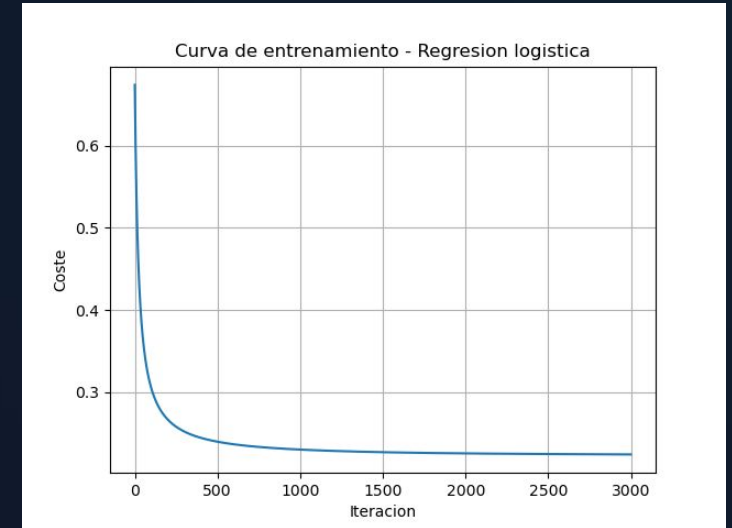
**3**

Escenarios de Balanceo

Enfoque en F1-Score para penalizar falsos positivos y negativos en contexto financiero.

# AJUSTE DE HIPERPARÁMETROS

- Los hiperparámetros se seleccionaron empíricamente a través de experimentación preliminar, evaluando el rendimiento del modelo en el conjunto de validación
- Se utilizaron las curvas de entrenamiento para evaluar cómo y cuándo convergen los modelos.



# DIAGNÓSTICO DE LOS MODELOS

## Curvas de Aprendizaje

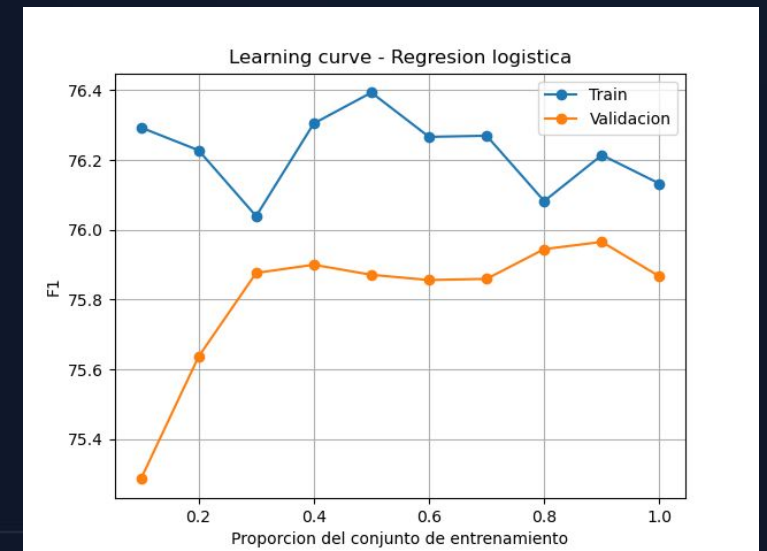
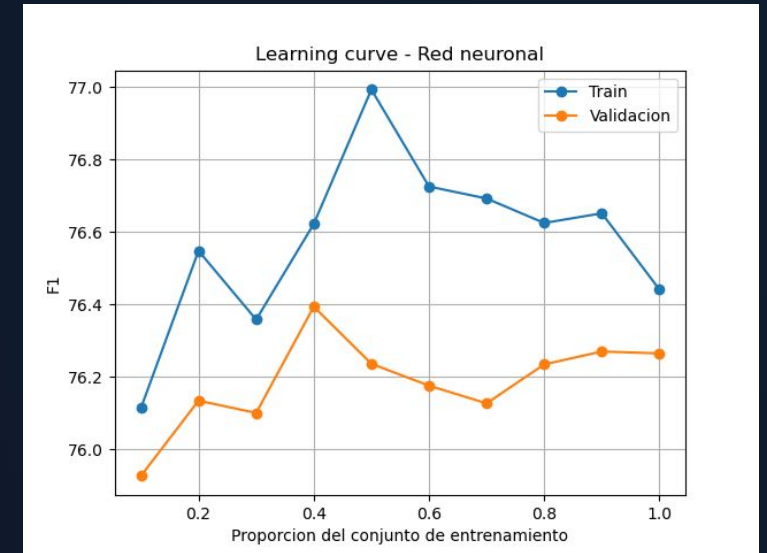
Las curvas muestran convergencia rápida.



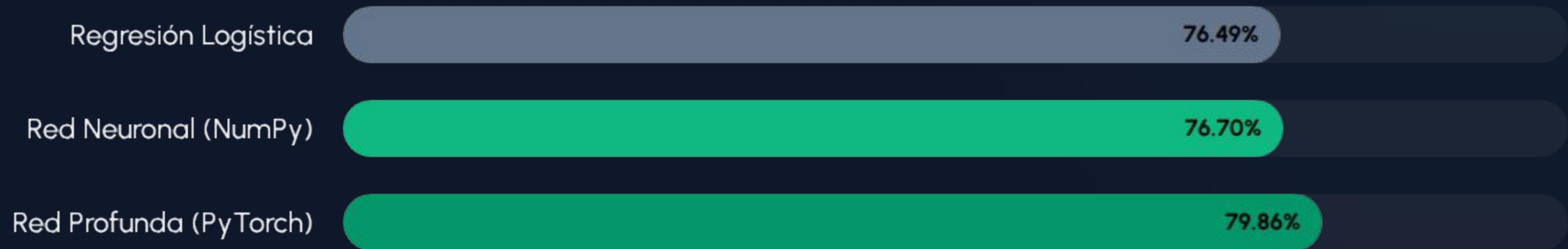
**Techo de cristal:** El rendimiento se estabiliza con una fracción del dataset, más datos no mejoran el resultado.



**Sesgo vs Varianza:** Error de train y val muy cercanos (Bajo Overfitting).

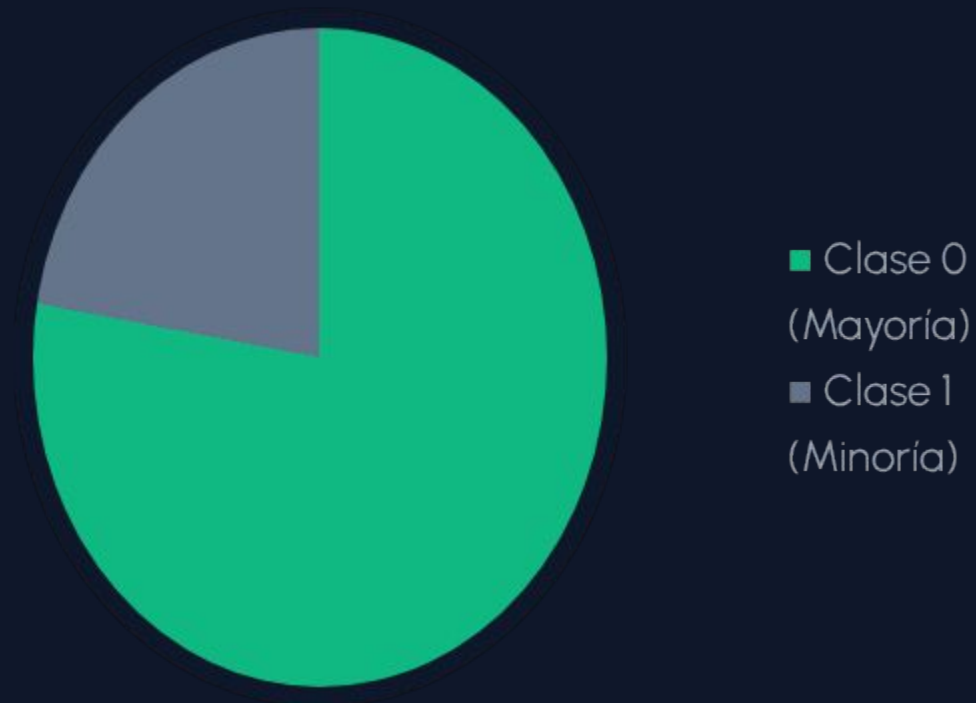


# RESULTADOS COMPARATIVOS (TEST)



*Métrica: F1-Score en conjunto de Test (Sin Balanceo)*

# TRATAMIENTO DEL DESBALANCEO



## Impacto de Técnicas

-  **Undersampling:** Sube Recall (92%) pero desploma Precision.
-  **SMOTE:** Resultados similares a undersampling; no aporta info nueva.
-  **Original:** Mejor F1 global.

# UNDERSAMPLING VS OVERSAMPLING

## Under

| Modelo         | Precision | Recall | F1     |
|----------------|-----------|--------|--------|
| Reg. Logística | 61.22%    | 92.05% | 73.54% |
| NN NumPy       | 61.16%    | 92.65% | 73.68% |
| DNN PyTorch    | 64.83%    | 89.30% | 75.12% |

## SMOTE

| Modelo         | Precision | Recall | F1     |
|----------------|-----------|--------|--------|
| Reg. Logística | 61.98%    | 91.70% | 73.97% |
| NN NumPy       | 61.27%    | 92.30% | 73.65% |
| DNN PyTorch    | 69.49%    | 87.10% | 77.30% |

## | ANÁLISIS CRÍTICO Y CONCLUSIÓN



- // *El problema tiene una estructura casi lineal. La Regresión Logística ya captura la esencia predictiva, haciendo que la complejidad de las redes neuronales apenas sea rentable.*

# ¿Preguntas?

Muchas gracias por su atención.

Github: `loan_approval`

---